



QuartierConnect

« Le lien qui rapproche votre quartier »

Proposition de projet — Connected Neighbours

Validation du sujet — Étape 1 · 15 mars 2026

Composition du groupe

Claudio REIBAUD	Chef de projet & Fullstack Dev
Andras SCHULLER	Front-end & Documentation
Mouhamadou N'DIAYE	Infrastructure & DevOps

Enseignant référent

Frédéric SANANES sananes@myges.fr



Table des matières

Résumé exécutif	2
1 Présentation du projet	3
1.1 Contexte	3
1.2 Objectifs	3
1.3 Public cible	3
2 Description fonctionnelle de l'application	4
2.1 Vue d'ensemble	4
2.2 Fonctionnalités principales	4
2.2.1 Modélisation géographique du quartier	4
2.2.2 Services entre voisins	4
2.2.3 Signature numérique sécurisée	5
2.2.4 Événements et recommandations Neo4j	5
2.2.5 Messagerie multimédia	5
2.2.6 Votes	5
2.2.7 Sécurité et RGPD	6
2.2.8 Multilingue	6
2.2.9 Application Java Desktop	6
2.3 Parcours utilisateur	7
3 Architecture et infrastructure technique	8
3.1 Architecture générale	8
3.2 Technologies utilisées	9
3.3 Gestion des données	10
3.4 Mécanisme de synchronisation <i>Offline-First</i>	12
4 Organisation du projet	13
4.1 Gestion du code source	13
4.2 Gestion de projet	13
4.3 Répartition des rôles	14
5 Identité du projet	16
5.1 Nom du projet	16
5.2 Logo et identité visuelle	16
Conclusion	17



Résumé exécutif

QuartierConnect est une plateforme numérique collaborative destinée aux habitants d'un quartier. Elle repose sur un **site web React** accessible à tous les résidents et sur une **application desktop Java** réservée aux administrateurs.

L'ensemble est conteneurisé via **Docker Compose** (7 services) et piloté par un back-end **Node.js + NestJS**, trois bases de données (**MongoDB**, **Neo4j** et **PostgreSQL**) et un micro-langage d'interrogation développé avec **PLY (Python Lex-Yacc)**, intégré au back-end via **pythonia**.

Les fonctionnalités couvrent : l'échange de services entre voisins (système de points, contrats **PDF auto-générés**), la signature numérique sécurisée SHA-256 avec **MFA** obligatoire, les événements communautaires avec recommandations par graphe social, la messagerie multimédia **WebSocket**, les votes paramétrables en quatre types et la gestion administrative en mode **offline-first**.

Le projet répond à l'intégralité du cahier des charges *Connected Neighbours* et intègre toutes les contraintes imposées (**React**, **Java**, **MongoDB**, **Neo4j**, **SQLite**, **Docker**, **micro-langage**). Les choix complémentaires de l'équipe (NestJS, intégration PLY via **pythonia**, ajout de **PostgreSQL** pour la synchronisation admin) sont documentés et justifiés dans ce document.

1 Présentation du projet

1.1 Contexte

Les quartiers résidentiels manquent d'outils numériques adaptés pour structurer l'entraide locale, formaliser les échanges et gérer les affaires communautaires. Les solutions généralistes (réseaux sociaux, groupes de messagerie) ne répondent pas à ces besoins de manière sécurisée et organisée.

QuartierConnect s'inscrit dans le cadre du projet académique *Connected Neighbours* (3AL2) et propose une plateforme dédiée, géolocalisée et sécurisée, disponible sur navigateur et sur poste de travail.

1.2 Objectifs

- Permettre l'**échange de services** entre voisins, valorisé par un système de points.
- Offrir un outil de **signature numérique** sécurisée (SHA-256 + MFA).
- Faciliter l'**organisation d'événements** avec recommandations personnalisées via Neo4j.
- Proposer une **messagerie multimédia** en temps réel (WebSocket).
- Fournir aux administrateurs un outil de **gestion desktop offline-first**.
- Respecter les exigences **RGPD** : export, suppression, consentement.

1.3 Public cible

Profil	Usage
Habitant	Accès complet au site web : services, événements, messagerie, votes, signature de documents.
Modérateur	Mêmes droits que l'habitant, plus la suppression de tout contenu inapproprié.
Administrateur	Gestion des quartiers, des utilisateurs, accès au back-office web et à l'application Java Desktop.

2 Description fonctionnelle de l'application

2.1 Vue d'ensemble

L'application est divisée en deux périmètres complémentaires :

- **Site web (React)** — accessible à tous les résidents depuis un navigateur. Regroupe toutes les fonctionnalités collaboratives.
- **Application desktop Java** — réservée aux administrateurs. Gestion des incidents, statistiques et alertes. Fonctionne hors-ligne avec synchronisation automatique au retour de la connexion.

2.2 Fonctionnalités principales

2.2.1 Modélisation géographique du quartier

Un administrateur délimite un quartier en dessinant un polygone sur une carte **OpenStreet-Map** via `Leaflet.js` et son plugin `Leaflet.draw`. Les coordonnées sont stockées en **GeoJSON** dans **MongoDB** (index géospatial `2dsphere`). À chaque création ou modification, le back-end détecte automatiquement les chevauchements via `$geoIntersects`. Chaque habitant appartient à un unique quartier et n'interagit qu'avec ses voisins directs.

2.2.2 Services entre voisins

Les habitants publient des **annonces d'offre ou de demande** de service (titre, description, catégorie, type gratuit/payant, durée estimée).

Système de points :

- 1 heure rendue = **2 points**; action rapide (<30 min) = **1 point**.
- Coefficient multiplicateur par catégorie, configurable par l'administrateur.
- Solde négatif toléré jusqu'à **-10 points**.
- Historique complet des transactions consultable par l'habitant.

Contrats automatiques : tout service payant déclenche la génération d'un contrat PDF (identités des parties, description, points, date, zones de signature). Les points sont transférés uniquement après double signature.

2.2.3 Signature numérique sécurisée

1. Import d'un PDF ou réception d'un contrat auto-généré.
2. Placement des **zones de signature** par glisser-déposer (`react-pdf-viewer`).
3. **MFA TOTP** obligatoire avant toute signature.
4. La signature (canvas `react-signature-canvas`) est associée à un **hash SHA-256** du document et à l'identité de l'utilisateur.
5. Horodatage côté serveur; métadonnées stockées séparément du PDF dans **MongoDB GridFS** (immuables).
6. **Journal d'audit immuable** : import, invitation, signature, consultation — chaque action est tracée.
7. Statuts: `draft` → `partial` → `fully_signed` . Les deux parties sont notifiées à chaque transition.

2.2.4 Événements et recommandations Neo4j

Tout habitant crée un événement (titre, date, lieu, capacité, catégorie). **Interface swipe** (style Tinder) : `droite = Intéressé`, `gauche = Pas intéressé` . Les interactions alimentent le moteur **Neo4j** qui suggère des événements, services et voisins selon la proximité sociale et les affinités passées.

2.2.5 Messagerie multimédia

- Chat 1-à-1 et de groupe en temps réel via **WebSocket**.
- Photos (JPEG/PNG, max 5 Mo), vidéos (MP4, max 1 min) et messages vocaux (MP3, max 2 min) stockés dans **MongoDB GridFS**.
- Indicateur « en train d'écrire... » et statut online/offline en temps réel.
- Chiffrement en transit : HTTPS + WSS.
- *Bonus* : appels vidéo pair-à-pair via **WebRTC**.

2.2.6 Votes

Type	Description
Binaire	Oui / Non
Choix unique	Une option parmi N
Choix multiple	Plusieurs options sélectionnables
Vote pondéré	Points à répartir librement entre les options

Paramètres configurables : durée, anonymat, quorum, visibilité des résultats, restriction à un groupe. Architecture **Pattern Strategy** : chaque type est un module indépendant, extensible sans toucher au code existant.

2.2.7 Sécurité et RGPD

Authentification :

- Email + mot de passe hashé **bcrypt** (10 rounds).
- **MFA TOTP** obligatoire : connexion initiale, modification de données sensibles (email, mot de passe, téléphone), acte de signature.
- Tokens **JWT HS256** : access token (15 min) + refresh token (7 jours).
- **SSO partagé** web ↔ Java : le même compte est utilisé sur les deux plateformes.

RGPD :

- Droit d'accès : `GET /api/me/export` → JSON de toutes les données.
- Droit à l'oubli : `DELETE /api/me/delete` → suppression + anonymisation dans Neo4j.
- Droit de rectification : modification du profil à tout moment.
- Consentement explicite à l'inscription (timestamp enregistré).
- Données stockées uniquement sur serveurs UE.

2.2.8 Multilingue

Support **français** et **anglais** via `i18next` + `react-i18next`. Détection automatique par le navigateur, sélection manuelle dans le profil. Architecture extensible (ajout d'un fichier JSON par locale).

2.2.9 Application Java Desktop

Réservée aux administrateurs :

- **Incidents et alertes** : TableView JavaFX, tri/filtrage par statut et date, ajout et modification d'incidents (disponible en offline).
- **Statistiques** : événements créés, services échangés, messages, votes — BarChart, LineChart, PieChart, filtres par période et habitant.
- **Plugins extensibles** : export CSV, export PDF de rapports, analyse du graphe social, calendrier local des événements.
- **Offline-first** : SQLite embarqué, synchronisation automatique toutes les 30 s, stratégie last-write-wins, log des conflits.
- SSO JWT partagé avec le site web, mises à jour automatiques depuis le serveur central, désinstallation depuis l'UI, thèmes CSS JavaFX dynamiques.
- Livré sous forme d'un **JAR auto-exécutable** (Maven Shade Plugin).



2.3 Parcours utilisateur

Habitant — échange de service :

Inscription → MFA setup → connexion → publication d'une annonce → acceptation par un voisin → contrat PDF généré → double signature (MFA requis) → transfert de points.

Habitant — événement :

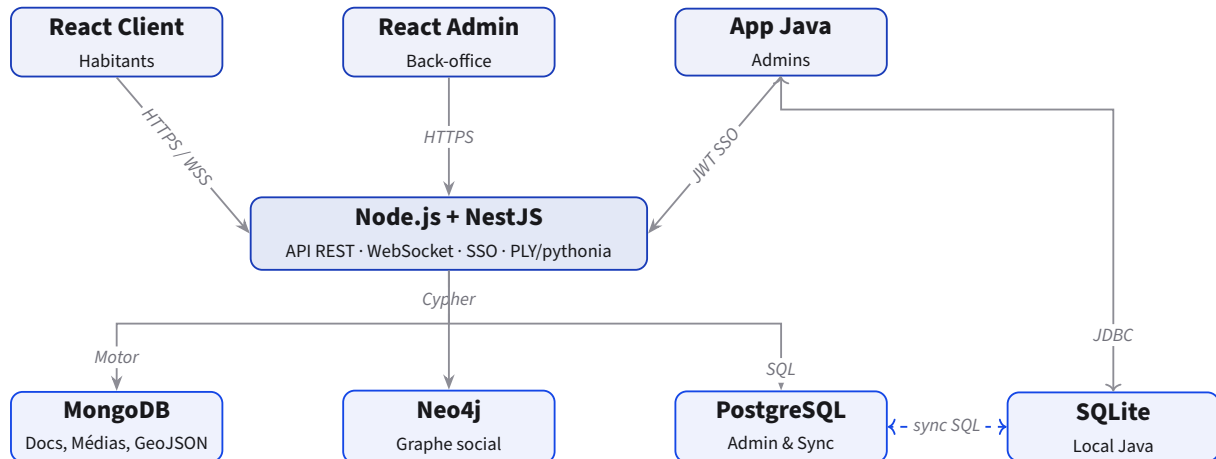
Page swipe → swipe droite → inscription confirmée → Neo4j affine les recommandations.

Administrateur — application Java :

Connexion SSO → consultation des incidents → modification hors-ligne si nécessaire → synchronisation automatique → export rapport PDF ou CSV via plugin.

3 Architecture et infrastructure technique

3.1 Architecture générale



7 conteneurs Docker Compose :

#	Conteneur	Port(s)
1	Caddy — reverse proxy HTTPS	80, 443
2	React Client	3000
3	React Admin	3001
4	NestJS (Node.js API)	3000
5	MongoDB	27017
6	Neo4j	7474, 7687
7	PostgreSQL	5432

Trois environnements distincts : `dev` (hot reload), `test` (base isolée, seed automatique), `prod` (HTTPS Caddy, images pré-buildées).

3.2 Technologies utilisées

Composant	Technologie	Justification détaillée
Back-end	NestJS (Node.js)	Framework Node.js modulaire et opinionné, TypeScript natif, décorateurs injectables et génération Swagger automatique via <code>@nestjs/swagger</code> .
Front-end	React 19 + Vite	Framework UI performant avec support natif des <i>Server Components</i> et du HMR.
Data Fetching	TanStack Query	Gestion du cache serveur, synchronisation automatique et <i>Optimistic Updates</i> .
Routage	TanStack Router	Système de navigation 100% type-safe avec gestion granulaire des <i>search params</i> .
Gestion Form	TanStack Form + Zod	Validation de schémas robuste et gestion d'état de formulaire haute performance.
Data Grids	TanStack Table v8	Logique <i>headless</i> pour manipuler les listes d'habitants : tri, filtres et facettes.
Design System	Shadcn + Tailwind v4	Bibliothèque de composants accessibles (Radix UI) avec moteur CSS ultra-rapide.
Client lourd	Java 25 + JavaFX	Application d'administration robuste avec gestion du multithreading native.
BDD Documents	MongoDB	Stockage NoSQL flexible pour les dossiers d'habitants et les métadonnées de documents.
Graphe Social	Neo4j	Moteur de graphe performant pour cartographier les liens d'entraide entre voisins.
BDD Admin	PostgreSQL	Base relationnelle serveur gérant les données d'administration et facilitant la synchronisation SQL avec le client Java.
BDD Offline	SQLite (JDBC)	Cache transactionnel local de l'application Java Desktop, miroir SQL partiel des données PostgreSQL pour le mode hors-ligne.
Temps réel	NestJS Gateways	Gestion native des WebSockets via les <i>Gateways</i> NestJS pour la messagerie instantanée en temps réel.
Cartographie	Leaflet + OSM	Visualisation géographique open source permettant le tracé dynamique de zones.
Signature PDF	pdf-lib + Signature	Signature manuscrite sur canvas et injection de certificats dans les fichiers PDF.
Sécurité	JWT + TOTP (MFA)	Sécurisation des sessions par jetons signés et double authentification par QR Code.
i18n	i18next	Internationalisation complète avec détection de langue et <i>lazy loading</i> des traductions.

Composant	Technologie	Justification détaillée
Analyse DSL	PLY + pythonia	Micro-langage Python (Lex/Yacc) générant l'AST d'interrogation, exposé au back-end NestJS via <code>pythonia</code> (interop JS/Python zéro-copie).
Tests	Jest + Playwright	Suite de tests unitaires et E2E garantissant la non-régression sur tous les navigateurs.

L'architecture retenue s'appuie sur trois choix complémentaires qui méritent justification :

- 1. NestJS comme back-end principal :** `NestJS` est un framework Node.js modulaire et opinionné, écrit en TypeScript natif. Son système de décorateurs, ses modules injectables et la génération Swagger automatique (via `@nestjs/swagger`) garantissent une API robuste, maintenable et documentée. Ses `Gateways WebSocket` intégrées gèrent nativement les connexions persistantes pour la messagerie en temps réel.
- 2. Micro-langage PLY intégré via `pythonia` :** Le cahier des charges impose un micro-langage Lex/Yacc. `PLY (Python Lex-Yacc)` reste l'implémentation la plus fidèle et robuste de cette spécification. Plutôt que de réécrire le compilateur en JavaScript, nous utilisons `pythonia` — une bibliothèque Node.js qui expose nativement des modules Python en tant qu'objets JavaScript (zéro copie, interopérabilité synchrone). Le micro-langage Python est ainsi appelé directement depuis les services NestJS, sans processus séparé ni surcoût d'infrastructure.
- 3. PostgreSQL pour la synchronisation admin :** L'ajout de `PostgreSQL` côté serveur répond à un besoin fonctionnel précis : faciliter la synchronisation avec la base `SQLite` embarquée dans l'application Java Desktop. Les deux moteurs partageant la même sémantique SQL, le schéma de données admin est rigoureusement identique, réduisant la logique de mapping à une simple copie différentielle. PostgreSQL gère l'intégralité des données d'administration (incidents, statistiques, alertes) qui constituent le cœur métier de l'application Desktop.

3.3 Gestion des données

MongoDB (Base principale imposée)

Cœur opérationnel de notre architecture, MongoDB s'intègre avec notre stack Node.js de manière fluide et bidirectionnelle, grâce à son format documentaire natif et au driver officiel `Mongoose`. Ses index géospatiaux `2dsphere` permettent de résoudre nativement les problématiques de délimitation de quartiers et d'appartenance (`$geoIntersects`) sans nécessiter d'extension complexe (type PostGIS). Enfin, le moteur **GridFS** offre une solution robuste et unifiée pour le stockage transactionnel des fichiers lourds (PDF des contrats, médias de la messagerie), évitant ainsi d'alourdir l'infrastructure avec un service de stockage de type S3.

Pourquoi PostgreSQL est-il ajouté et comment se synchronise-t-il avec SQLite ?

L'introduction de PostgreSQL côté serveur répond à un argument fonctionnel central : la synchronisation avec la base SQLite locale du client Java. Voici les points clés de cette décision :

- **Symétrie SQL** : PostgreSQL et SQLite partagent la même sémantique SQL standard. Le schéma des tables d'administration (incidents, alertes, statistiques) est défini une seule fois et déployé sur les deux moteurs sans transformation, éliminant tout risque de désérialisation complexe.
- **Séparation claire des responsabilités** : PostgreSQL est le miroir serveur exclusif des données d'administration, tandis que MongoDB conserve son rôle de source principale pour les données communautaires (profils, services, événements, médias, GeoJSON). Les deux bases sont complémentaires et non redondantes.
- **Synchronisation simplifiée** : La réconciliation exploite directement les primitives SQL communes — `INSERT OR REPLACE` côté SQLite et `ON CONFLICT DO UPDATE` côté PostgreSQL. La file d'attente de mutations (`sync_queue`) est résolue par comparaison de timestamps sans convertisseur de format supplémentaire.
- **Impact infrastructure maîtrisé** : L'ajout du conteneur PostgreSQL est justifié par le gain fonctionnel concret. Sa consommation de ressources est bien inférieure à celle de MongoDB et il n'alourdit pas significativement les pipelines CI/CD.

Neo4j (Graphe social imposé)

Modélise nativement la topologie du réseau d'habitants (services échangés, événements communs, messagerie). Le langage Cypher offre des performances de parcours de graphe inégalées pour le moteur de recommandation et le calcul de proximité sociale, là où des requêtes SQL relationnelles s'effondreraient en temps de réponse.

SQLite (Base locale Java imposée)

Moteur relationnel ultra-léger embarqué via JDBC, pilier de notre architecture *offline-first* pour l'application Desktop. Il agit comme un cache transactionnel intelligent pour stocker les incidents locaux et la file d'attente de synchronisation (*sync queue*) lors des coupures réseau. Son schéma SQL est conçu en miroir de **PostgreSQL**, le pendant serveur dédié à l'administration, ce qui réduit la logique de réconciliation à une simple copie différentielle.

3.4 Mécanisme de synchronisation *Offline-First*

L'application client lourd Java repose sur un pattern architectural **offline-first**, garantissant une résilience totale de l'administration face aux pertes de réseau. Le moteur `SQLite` agit comme cache transactionnel local avec la stratégie de réconciliation suivante :

- **Persistance locale (Optimistic Update)** : Toute mutation (création, modification, résolution d'incident) est immédiatement validée en `SQLite` avec un marqueur d'état (`is_dirty = true`) et un horodatage précis, garantissant un retour UI instantané sans latence réseau.
- **Sonde réseau (Worker Thread)** : Un processus d'arrière-plan asynchrone vérifie la connectivité à l'API NestJS via un mécanisme de *polling* léger à intervalle régulier (**30 secondes**).
- **Phase de Push (Upstream)** : Dès le rétablissement de la connexion, la file d'attente des mutations (*sync queue*) est dépilée. Les enregistrements marqués `is_dirty` sont envoyés à l'API, idéalement par lots (*batching*) pour limiter les appels réseau.
- **Phase de Pull (Downstream)** : Le client interroge le serveur pour récupérer le *delta* des modifications survenues depuis son dernier horodatage (`last_sync_timestamp`), optimisant la bande passante.
- **Résolution des conflits (LWW)** : Le système applique une politique de **Last-Write-Wins**. En cas de collision concurrentielle sur une même ressource, l'API tranche en comparant le timestamp de la *payload* entrante avec celui de l'enregistrement cible dans PostgreSQL.
- **Audit et Traçabilité** : Chaque collision résolue et chaque erreur de synchronisation sont consignées dans une table locale `sync_log`, offrant un historique transparent des réconciliations.
- **Réactivité UI** : Le thread principal (*JavaFX Application Thread*) est notifié à la fin du cycle de synchronisation, déclenchant un rafraîchissement non-bloquant des vues (tableaux de bord, listes d'incidents).

4 Organisation du projet

4.1 Gestion du code source

Dépôt **GitHub** : `github.com/creibaud/QuartierConnect`

<code>backend/</code>	API Node.js + NestJS
<code>dsl/</code>	Micro-langage PLY (Python) intégré via pythonia
<code>frontend/</code>	Monorepo React Client + Admin
<code>frontend/apps/client/</code>	Front-end React Client (habitants)
<code>frontend/apps/admin/</code>	Front-end React Admin (back-office)
<code>java-app/</code>	Application Java Desktop
<code>docker/</code>	Docker Compose, Caddyfile, scripts
<code>docs/</code>	Documentation, schémas, Figma exports

Pourquoi un monorepo pour les applications React ?

Les deux applications React partagent une même base de composants UI (Shadcn) et une configuration Vite commune.

Un **monorepo** (avec `pnpm Workspaces`) permet de mutualiser les dépendances et d'assurer une cohérence visuelle et fonctionnelle entre le client grand public et le back-office, tout en gardant une séparation claire du code métier spécifique à chaque périmètre et deux pipelines de build distincts (client : port 3000, admin : port 3001).

Stratégie de branches (Git Flow simplifié) :

<code>main</code>	Code stable — merges uniquement aux étapes de rendu
<code>develop</code>	Branche d'intégration commune
<code>feat/backend</code>	Développement back-end
<code>feat/dsl</code>	Développement du micro-langage PLY (Python)
<code>feat/frontend-client</code>	Développement front-end
<code>feat/frontend-admin</code>	Développement back-office
<code>feat/docker</code>	Infrastructure Docker
<code>feat/java</code>	Application Java
<code>feat/docs</code>	Documentation et design

4.2 Gestion de projet

Suivi via **Trello** : `trello.com/b/oidFFT0p/pa-quartierconnect`

Structure du Trello : une colonne par étape du projet, puis En cours / Review / Terminé / Bloqué. Chaque carte contient : responsable, dates d'échéance, checklist des sous-tâches, label catégorie.

Planning prévisionnel :

Étape	Période	Objectif
Étape 1	jusqu'au 15 mars 2026	Validation du sujet, proposition, infrastructure, technologies, nom, logo.
Étape 2	16 mars → 5 avril 2026	30% réalisé. Pages web, API de base, démo Java, tests, Figma, docs. <i>Réunion 08/04.</i>
Étape 3	6 avril → 31 mai 2026	60% réalisé. API complète + Swagger, client web, Java avec sync. <i>Réunion 04/06.</i>
Étape 4	1er juin → 28 juin 2026	95% réalisé. React Admin, micro-langage PLY finalisé. <i>Réunion 02/07.</i>
Rendu final	1er → 19 juillet 2026	Sources nettoyés, JAR, BDD, dossier technique, guide, soutenance 30 min.

4.3 Répartition des rôles

Membre	Rôle	Responsabilités
Claudio REIBAUD	Chef de projet & Fullstack Dev	Principales : Back-end core et avancé (<code>NestJS</code>), Front-end (<code>React</code> Client & Admin), et application Desktop <code>Java 25 / JavaFX</code>. Secondaires : Écriture du micro-langage (<code>PLY / pythonia</code>), maquettage UI/UX, orchestration Docker, renfort sur les tests et la documentation.
Andras SCHULLER	Front-end & Documentation	Principales : Couverture de tests (<code>Jest/Playwright</code>), rédaction de la documentation (dossier technique, UML, guide utilisateur) et développement Front-end React. Secondaires : Maquettage UI/UX (<code>Figma</code>) et définition de la syntaxe du micro-langage (<code>PLY</code>).
Mouhamadou N'DIAYE	Infrastructure & DevOps	Principales : Architecture Infra (VPS, <code>Caddy</code>), conteneurisation (<code>Docker Compose</code>) et pipelines CI/CD (<code>GitHub Actions</code>). Secondaires : Développement des endpoints API CRUD côté Back-end et renfort sur les tests automatisés.



La répartition des tâches suit une logique d'entonnoir dynamique. Chaque membre est responsable en priorité de son périmètre **principal**. Une fois ses livrables prioritaires achevés (ou en attente de validation), le membre bascule sur ses tâches **secondaires** afin de désengorger les goulots d'étranglement de l'équipe. Cette flexibilité garantit que les postes critiques (comme l'API ou les tests) ne bloquent jamais l'avancement global du projet.

5 Identité du projet

5.1 Nom du projet

QuartierConnect — nom choisi pour sa clarté et son accessibilité. Il exprime directement la vocation de la plateforme : *connecter* les habitants d'un même *quartier*. Le slogan associé — « *Le lien qui rapproche votre quartier* » — renforce la dimension humaine et de proximité du projet.

5.2 Logo et identité visuelle



Lecture du logo :

Le logotype met en scène trois foyers disposés de manière circulaire et reliés par un maillage de nœuds. Cette géométrie symbolise la création de lien social de proximité et l'entraide entre voisins. Visuellement, cette structure en réseau évoque la topologie en graphe de notre base **Neo4j**, soulignant l'ADN technologique du projet.

Palette de couleurs & Design System (Shadcn UI) :

L'interface repose sur le système de composants `Shadcn/ui` (thème *Zinc / Blue*). Ce choix garantit un rendu moderne, une excellente accessibilité (Radix UI) et un confort visuel optimal.

Token	Light	Dark	Usage
Background	 #FFFFFF	 #09090B	Fond de l'interface, surface principale des pages.
Primaire	 #1D4ED8	 #193CB8	Boutons d'action, liens et mise en évidence des points.
Secondaire	 #2563EB	 #1447E6	États de survol, badges et éléments graphiques interactifs.
Muted	 #71717A	 #A1A1AA	Textes secondaires, placeholders et bordures fines.



Conclusion

QuartierConnect couvre l'intégralité du cahier des charges *Connected Neighbours* : plateforme web collaborative, application desktop Java offline-first, graphe social Neo4j, micro-langage PLY intégré via pythonia, signature numérique SHA-256 avec MFA, conteneurisation Docker en 7 services et conformité RGPD complète. Le projet sera suivi rigoureusement via **Trello** et **GitHub**, conformément aux exigences du syllabus.